Univerzitet u Beogradu - Elektrotehnički fakultet

Katedra za računarsku tehniku i informatiku

DIPLOMSKI RAD

Projektovanje informacionog sistema za upravljanje klinikama

Kandidat
Marko Perunović, br. indeksa 2019/0280

Mentor
vanr. prof. dr Miloš Cvetanović

Beograd, april 2026. godine

REZIME RADA

Modernizacija poslovanja u medicinskim ustanovama zahtijeva softverska rješenja koja prevazilaze puku evidenciju pacijenata i obezbjeđuju pouzdanu podršku za finansijsko upravljanje. Ovaj diplomski rad bavi se projektovanjem i implementacijom serverskog sistema za upravljanje klinikama u okviru softverskog rješenja zasnovanog na modelu pružanja usluge (SaaS), sa posebnim fokusom na integritet finansijskih transakcija i bezbjednost podataka u višekorisničkom (multi-tenant) okruženju.

Analizom postojećih rješenja identifikovani su ključni problemi koji se javljaju u praksi, uključujući nekonzistentnost podataka usljed paralelnog rada više operatera, nedostatak pouzdanog revizorskog traga i neadekvatnu izolaciju podataka između različitih zakupaca sistema. Predloženo rješenje zasniva se na slojevitoj softverskoj arhitekturi, implementiranoj korišćenjem programskog jezika Python i Flask okvira, uz upotrebu PostgreSQL baze podataka za perzistenciju podataka.

Glavni doprinos rada ogleda se u implementaciji algoritama za automatsku raspodjelu uplata po principu vodopada, kao i u primjeni mehanizama pesimističkog zaključavanja na nivou redova baze podataka radi obezbjeđivanja konkurentnog pristupa zajedničkim resursima bez narušavanja ACID svojstava transakcija. Dodatno, realizovan je sistem za kriptografsku verifikaciju integriteta zapisa u audit logu, kao i mehanizam upravljanja sesijama zasnovan na verzionisanju autentifikacionih tokena. Kao rezultat, razvijen je pouzdan, skalabilan i bezbjedan sistem koji obezbjeđuje tačnost finansijskih podataka i smanjuje mogućnost zloupotreba u realnom vremenu.

Ključne riječi: multi-tenant arhitektura, integritet finansijskih transakcija, konkurentni pristup, pesimističko zaključavanje, SaaS, audit log.

SADRŽAJ

1 UVOD 1

2 TEORIJSKE OSNOVE I KORIŠĆENE TEHNOLOGIJE 3

2.1 SaaS i multi-tenant arhitektura 3

2.2 Integritet finansijskih sistema i ACID principi 5

2.3 Konkurentni pristup i upravljanje transakcijama 6

2.4 Pregled korišćenih tehnologija 7

3 ANALIZA ZAHTJEVA I MODELOVANJE SISTEMA 9

3.1 Funkcionalni zahtjevi 9

3.2 Nefunkcionalni zahtjevi 10

3.3 Model podataka i entitetsko-relacioni dijagram 11

3.4 Mockup ekrani korisničkog interfejsa 15

4 ARHITEKTURA I IMPLEMENTACIJA SISTEMA 19

4.1 Opšti arhitektonski pregled sistema 19

4.2 Sloj servisa i upravljanje poslovnom logikom 20

4.3 Upravljanje transakcijama i konkurentni pristup 21

4.4 Upravljanje finansijskim tokovima i dnevno zatvaranje 22

4.5 Revizorski trag i bezbjednosni mehanizmi 27

5 VERIFIKACIJA I TESTIRANJE SISTEMA 29

5.1 Strategija testiranja 29

5.2 Jedinični i integracioni testovi 30

5.3 Testiranje bezbjednosnih mehanizama 30

5.4 Ograničenja testnog okruženja i razlika između testne i produkcione baze 31

5.5 Rezultati testiranja 31

6 ZAKLJUČAK 33

7 LITERATURA 35

1 UVOD

Savremeno poslovanje medicinskih ustanova, posebno manjih i srednjih privatnih klinika, sve više zavisi od informacionih sistema koji podržavaju ne samo administrativne procese, već i finansijsko poslovanje. Iako na tržištu postoji veliki broj softverskih rješenja za evidenciju pacijenata i zakazivanje pregleda, značajan dio tih sistema ne pruža adekvatnu podršku za precizno, transparentno i bezbjedno upravljanje finansijskim tokovima. U praksi, finansijski procesi u klinikama često se oslanjaju na ručne evidencije, jednostavne tabelarne proračune ili nedovoljno kontrolisane softverske alate, što povećava rizik od grešaka, zloupotreba i nekonzistentnosti podataka.

Posebno problematične situacije nastaju u okruženjima gdje više zaposlenih istovremeno rukuje istim finansijskim podacima, kao što su naplate rata, djelimične uplate, kašnjenja u plaćanju i dnevno zatvaranje kase. U takvim uslovima, nedostatak mehanizama za kontrolu konkurentnog pristupa može dovesti do tzv. *race condition* scenarija, u kojima paralelne operacije rezultiraju netačnim finansijskim stanjima. Dodatni izazov predstavlja i potreba za jasnim razdvajanjem podataka između različitih klinika u okviru istog softverskog sistema, naročito u modelima zasnovanim na pružanju usluge (Software as a Service – SaaS).

U realnim uslovima rada, medicinske ustanove često koriste zajedničke računarske terminale na recepciji ili u administraciji, koje tokom dana koristi više zaposlenih. U takvom okruženju, učestalo odjavljivanje i ponovno prijavljivanje korisnika predstavlja značajan operativni teret i usporava radni proces. Istovremeno, određene finansijske operacije, poput odobravanja uplata, isplata napojnica ili zatvaranja kase, zahtijevaju viši nivo autorizacije. Zbog toga se javlja potreba za mehanizmima sekundarne autorizacije koji omogućavaju brzu, ali bezbjednu potvrdu osjetljivih operacija od strane ovlašćenih korisnika, bez narušavanja integriteta sistema.

Poseban izazov predstavlja i upravljanje gotovinom u klinici. Gotovinski tokovi moraju biti precizno evidentirani, a sredstva namijenjena klinici jasno razdvojena od ličnih primanja zaposlenih, poput napojnica. Pored toga, neophodno je obezbijediti pouzdan mehanizam dnevnog finansijskog zatvaranja, kojim se fizičko stanje kase upoređuje sa evidencijom u sistemu, uz mogućnost dodatne kontrole i odobravanja od strane nadređenog osoblja. Iz ugla bezbjednosti, finansijski sistemi zahtijevaju potpun revizorski trag svih transakcija i mogućnost naknadne provjere izmjena, kako bi se spriječile neovlašćene ili neprimijećene manipulacije podacima.

Cilj ovog diplomskog rada je razvoj serverskog softverskog sistema koji obezbjeđuje dosljedan integritet finansijskih podataka i bezbjedan konkurentni pristup u okviru

višekorisničke, multi-tenant arhitekture za upravljanje klinikama. Sistem je projektovan kao backend rješenje koje služi kao centralna tačka obrade svih finansijskih i administrativnih operacija, sa jasnim razdvajanjem poslovne logike, sloja pristupa podacima i interfejsa prema klijentima.

U okviru rada implementirani su mehanizmi za obradu finansijskih transakcija koji garantuju očuvanje ACID svojstava i sprječavaju nastanak nekonzistentnih stanja u uslovima paralelnog rada više korisnika. Poseban doprinos rada ogleda se u primjeni zaključavanja na nivou redova baze podataka, čime se eliminišu greške izazvane konkurentnim pristupom zajedničkim resursima, kao i u definisanju strogo kontrolisanih tranzicija stanja finansijskih operacija.

Dodatni doprinos predstavlja uvođenje formalizovanih bezbjednosnih mehanizama za kontrolu pristupa i autorizaciju osjetljivih operacija, uključujući višeslojni model autorizacije i centralizovano upravljanje sesijama korisnika. Razvijeni sistem omogućava potpunu sljedivost finansijskih tokova, auditabilnost svih promjena i jasnu izolaciju podataka između različitih zakupaca sistema. Na taj način, rad pruža praktično rješenje za implementaciju pouzdanih finansijskih sistema u realnom, produkcionom okruženju.

Rad je organizovan u nekoliko tematskih cjelina. Nakon uvodnog poglavlja, u drugom poglavlju prikazane su teorijske osnove i korišćene tehnologije relevantne za razumijevanje problema, uključujući principe SaaS i multi-tenant arhitekture, integritet finansijskih sistema (ACID), te konkurentni pristup i transakcijsko upravljanje. Treće poglavlje bavi se analizom zahtjeva i modelovanjem sistema, sa posebnim osvrtom na model podataka i entitetsko-relacioni dijagram. U četvrtom poglavlju detaljno je opisana arhitektura i implementacija sistema, sa fokusom na slojevitu organizaciju (data layer, servisni sloj i API sloj), finansijske tokove, dnevno zatvaranje i prateće mehanizme konzistentnosti. Peto poglavlje posvećeno je verifikaciji i testiranju sistema kroz jedinične i integracione testove, uz osvrt na ograničenja testnog okruženja u odnosu na produkcionu bazu. U šestom poglavlju dati su zaključci i smjernice za budući razvoj. Na kraju rada nalazi se spisak korišćene literature, kao i prilozi sa dodatnim materijalima.

2 TEORIJSKE OSNOVE I KORIŠĆENE TEHNOLOGIJE

U ovom poglavlju prikazane su teorijske osnove i tehnologije koje predstavljaju temelj za projektovanje i implementaciju razvijenog sistema za upravljanje klinikama. Poseban fokus stavljen je na koncepte koji su od ključnog značaja za razumijevanje arhitekture sistema, uključujući SaaS i multi-tenant pristup, integritet finansijskih podataka, transakcijsko upravljanje i konkurentni pristup zajedničkim resursima. Pored teorijskih osnova, predstavljene su i odabrane tehnologije koje su korišćene za realizaciju sistema, uz obrazloženje njihove uloge i značaja u okviru cjelokupnog rješenja.

U nastavku poglavlja najprije su obrađeni principi SaaS i multi-tenant arhitekture, kao i osnovni aspekti integriteta finansijskih sistema zasnovani na ACID svojstvima transakcija. Nakon toga razmotreni su problemi konkurentnog pristupa i mehanizmi upravljanja transakcijama, a zatim je dat pregled korišćenih tehnologija i njihov doprinos implementaciji pouzdanog i bezbjednog serverskog sistema.

2.1 SaaS i multi-tenant arhitektura

Model pružanja softvera kao usluge (Software as a Service – SaaS) predstavlja savremeni pristup distribuciji i korišćenju softverskih sistema, pri čemu se aplikacija izvršava na centralizovanoj serverskoj infrastrukturi i dostupna je korisnicima putem mreže. U ovom modelu, korisnici ne upravljaju instalacijom, održavanjem i ažuriranjem softvera, već pristupaju funkcionalnostima sistema kroz standardizovane interfejse, dok su operativni i infrastrukturni aspekti u nadležnosti provajdera usluge.

Jedan od ključnih izazova SaaS sistema jeste podrška za istovremeni rad većeg broja nezavisnih korisnika i organizacija, što se ostvaruje primjenom multi-tenant arhitekture. U multi-tenant okruženju, više zakupaca sistema dijeli istu aplikaciju i bazu podataka, pri čemu je neophodno obezbijediti strogu logičku izolaciju njihovih podataka i operacija. Svaki zakupac mora imati utisak da koristi potpuno nezavisan sistem, iako se fizički resursi dijele.

Postoji više pristupa implementaciji multi-tenant arhitekture, uključujući odvojene baze podataka po zakupcu, odvojene šeme unutar iste baze, kao i zajedničku šemu sa logičkom izolacijom podataka. Izbor odgovarajućeg pristupa zavisi od zahtjeva sistema u pogledu skalabilnosti, performansi, troškova i bezbjednosti. U kontekstu finansijskih sistema, gdje su

konzistentnost i kontrola pristupa od presudnog značaja, posebno je važno da mehanizmi izolacije budu pouzdani i dosljedno primijenjeni na svim nivoima aplikacije.

U okviru ovog rada primijenjen je pristup zajedničke šeme baze podataka uz logičku izolaciju podataka na nivou redova, gdje se svi zapisi povezuju sa konkretnim zakupcem sistema. Ovakav pristup omogućava efikasno korišćenje resursa, jednostavnije održavanje sistema i centralizovano upravljanje poslovnom logikom, uz uslov da se striktno sprovode mehanizmi kontrole pristupa i validacije konteksta zakupca pri svakoj operaciji.

Primjena SaaS modela u domenu upravljanja klinikama donosi značajne prednosti, uključujući dostupnost sistema sa različitih lokacija, lakšu skalabilnost u skladu sa rastom broja korisnika, kao i mogućnost brze nadogradnje funkcionalnosti bez prekida rada. Istovremeno, ovakav model nameće visoke zahtjeve u pogledu bezbjednosti, integriteta podataka i pouzdanosti sistema, što predstavlja osnovu za dalje razmatranje konkurentnog pristupa i upravljanja finansijskim transakcijama u nastavku rada.

2.2 Integritet finansijskih sistema i ACID principi

Integritet finansijskih sistema predstavlja osnovni zahtjev svakog softverskog rješenja koje upravlja novčanim tokovima, bez obzira na oblast primjene. U kontekstu informacionih sistema, integritet podataka podrazumijeva da su svi finansijski zapisi tačni, konzistentni i otporni na greške koje mogu nastati usljed sistemskih kvarova, grešaka korisnika ili paralelnog izvršavanja operacija. Poseban izazov predstavlja rad u višekorisničkom okruženju, gdje više operatera istovremeno inicira finansijske transakcije nad zajedničkim skupom podataka.

Za obezbjeđivanje pouzdanosti finansijskih sistema, savremene relacione baze podataka oslanjaju se na koncept transakcija i skup svojstava poznatih pod akronimom ACID (Atomicity, Consistency, Isolation, Durability). Ova svojstva definišu formalni okvir u kojem se transakcije izvršavaju i predstavljaju osnovu za očuvanje integriteta podataka u složenim i konkurentnim sistemima.

Atribut atomarnosti (Atomicity) podrazumijeva da se transakcija izvršava u potpunosti ili se uopšte ne izvršava. U slučaju greške tokom obrade finansijske operacije, sistem mora biti u stanju da poništi sve djelimične izmjene i vrati bazu podataka u prethodno konzistentno stanje. Ovaj princip je od posebnog značaja kod operacija koje obuhvataju više povezanih zapisa, kao što su raspodjela uplata, ažuriranje stanja dugovanja ili evidentiranje gotovinskih tokova.

Konzistentnost (Consistency) označava da se svaka uspješno završena transakcija mora završiti u stanju koje poštuje sva definisana pravila integriteta sistema. To uključuje ograničenja baze podataka, poslovna pravila i relacije između entiteta. U finansijskim sistemima, konzistentnost garantuje da ne može doći do nelogičnih stanja, poput negativnih iznosa tamo gdje oni nisu dozvoljeni ili neslaganja između povezanih finansijskih evidencija.

Izolacija (Isolation) se odnosi na način na koji se paralelno izvršavane transakcije međusobno “vide”. Cilj izolacije je da se obezbijedi da istovremeni rad više korisnika ne dovede do konflikata i nekonzistentnih rezultata. U nedostatku adekvatnih mehanizama izolacije, mogu se javiti problemi kao što su izgubljene izmjene, prljava čitanja ili nekonzistentni proračuni. U finansijskim sistemima, ovakvi scenariji su neprihvatljivi, jer mogu direktno uticati na tačnost novčanih iznosa i povjerenje u sistem.

Trajnost (Durability) garantuje da će jednom potvrđena transakcija trajno ostati zapisana u sistemu, čak i u slučaju pada aplikacije, nestanka električne energije ili drugih nepredviđenih događaja. Ovaj princip obezbjeđuje da finansijski podaci ne mogu biti izgubljeni nakon što su korisniku potvrđeni kao uspješno sačuvani.

Primjena ACID principa predstavlja neophodan, ali ne i dovoljan uslov za izgradnju pouzdanog finansijskog sistema. U složenim SaaS okruženjima, gdje se istovremeno izvršava veliki broj operacija nad zajedničkim resursima, neophodno je dodatno razmotriti mehanizme upravljanja konkurentnim pristupom i kontrolu izvršavanja transakcija. Upravo ovi aspekti biće detaljnije razmotreni u narednom poglavlju, sa posebnim osvrtom na mehanizme zaključavanja i sprječavanje konkurentnih konflikata u višekorisničkim sistemima.

2.3 Konkurentni pristup i upravljanje transakcijama

Konkurentni pristup predstavlja jednu od ključnih karakteristika savremenih serverskih sistema, u kojima više korisnika istovremeno pristupa zajedničkim resursima. U višekorisničkim aplikacijama, posebno onima koje upravljaju finansijskim podacima, neophodno je obezbijediti da paralelno izvršavanje operacija ne dovede do narušavanja integriteta sistema. Neadekvatno upravljanje konkurentnošću može rezultirati greškama koje je teško uočiti, ali koje imaju ozbiljne posljedice po tačnost i pouzdanost finansijskih evidencija.

Jedan od najčešćih problema u konkurentnim sistemima jeste pojava tzv. race condition scenarija, u kojima ishod izvršavanja zavisi od redoslijeda i vremenskog preklapanja paralelnih operacija. U finansijskom kontekstu, ovakve situacije mogu dovesti do višestrukog knjiženja iste uplate, nekorektnog smanjenja dugovanja ili pogrešnog izračunavanja stanja kase. Ovakvi problemi su posebno izraženi u sistemima gdje više operatera istovremeno obrađuje naplate ili vrši izmjene nad istim finansijskim zapisima.

Za upravljanje konkurentnim pristupom u relacijama baza podataka koriste se različiti mehanizmi, među kojima se najčešće razlikuju optimistički i pesimistički pristupi zaključavanju. Optimistički pristup polazi od pretpostavke da su konflikti rijetki, te dozvoljava paralelno izvršavanje transakcija uz naknadnu provjeru konzistentnosti prije potvrđivanja izmjena. Iako ovaj pristup može donijeti bolje performanse u određenim scenarijima, u finansijskim sistemima nosi značajan rizik, jer detekcija konflikta često dolazi prekasno, nakon što su već izvršeni proračuni nad netačnim podacima.

Nasuprot tome, pesimistički pristup zaključavanju podrazumijeva preventivno zaključavanje resursa prije njihove obrade, čime se onemogućava paralelan pristup istim podacima dok se transakcija ne završi. Ovakav pristup smanjuje mogućnost konkurentnih konflikata i obezbjeđuje determinističko ponašanje sistema, po cijenu potencijalnog smanjenja paralelizma. U domenu finansijskih sistema, gdje je tačnost važnija od maksimalnih performansi, pesimističko zaključavanje predstavlja prihvatljiv i često neophodan kompromis.

Upravljanje transakcijama u konkurentnim sistemima zahtijeva pažljiv izbor nivoa izolacije, kao i dosljednu primjenu transakcionih granica u svim operacijama koje mijenjaju stanje sistema. Posebno je važno da složene finansijske operacije, koje obuhvataju više povezanih izmjena, budu obrađene kao jedinstvene transakcije, kako bi se spriječilo nastajanje djelimičnih i nekonzistentnih stanja.

U kontekstu SaaS sistema za upravljanje klinikama, konkurentni pristup dodatno je otežan činjenicom da više korisnika može istovremeno raditi u okviru istog zakupca sistema, često koristeći zajedničke terminale. Zbog toga je neophodno da sistem implementira stroge mehanizme kontrole konkurentnosti na nivou baze podataka i poslovne logike, kako bi se obezbijedilo da svaka finansijska operacija bude izvršena na ispravan i predvidiv način. Ovi principi predstavljaju teorijsku osnovu za arhitektonska i implementaciona rješenja koja će biti detaljno razmotrena u nastavku rada.

2.4 Pregled korišćenih tehnologija

U cilju realizacije serverskog sistema koji ispunjava zahtjeve pouzdanosti, bezbjednosti i skalabilnosti, u okviru ovog rada odabran je skup savremenih tehnologija koje omogućavaju efikasan razvoj i stabilan rad softverskog rješenja u produkcionom okruženju. Izbor tehnologija zasnivan je na njihovoj zrelosti, podršci za rad sa transakcionim sistemima i mogućnosti precizne kontrole poslovne logike.

Kao programski jezik korišćen je Python, koji zbog svoje čitljivosti, bogatog ekosistema biblioteka i široke primjene u serverskim aplikacijama predstavlja pogodan izbor za razvoj kompleksnih poslovnih sistema. Python omogućava brz razvoj i jasnu implementaciju poslovnih pravila, što je od posebnog značaja u sistemima sa izraženim finansijskim i bezbjednosnim zahtjevima.

Za izgradnju web serverskog sloja korišćen je Flask okvir, lagani i fleksibilni web framework koji omogućava potpunu kontrolu nad arhitekturom aplikacije. Za razliku od monolitnih rješenja, Flask ne nameće striktne obrasce, što omogućava jasnu separaciju slojeva sistema i prilagođavanje arhitekture specifičnim potrebama aplikacije. Ovakav pristup je naročito pogodan za backend sisteme sa kompleksnom poslovnom logikom.

Perzistencija podataka realizovana je korišćenjem relacione baze podataka PostgreSQL, koja nudi naprednu podršku za transakcije, zaključavanje i očuvanje ACID svojstava. PostgreSQL je izabran zbog svoje pouzdanosti, podrške za konkurentni rad i mogućnosti precizne kontrole nad izvršavanjem transakcija, što je ključno za finansijske sisteme. U

razvojnim i testnim okruženjima korišćena je SQLite baza podataka, zbog jednostavne konfiguracije i brzog pokretanja.

Za rad sa bazom podataka korišćen je objektno-relacioni mapirajući okvir SQLAlchemy u verziji 2.0, koji omogućava savremen i eksplicitan stil pisanja upita, kao i jasnu kontrolu nad transakcionim kontekstom. SQLAlchemy omogućava kombinovanje visokonivou apstrakcija sa mogućnošću direktne kontrole nad SQL operacijama, što je posebno važno u sistemima koji zahtijevaju precizno upravljanje konkurentnim pristupom.

Validacija i serijalizacija podataka realizovana je korišćenjem biblioteke Pydantic, koja omogućava strogu provjeru strukture i tipova ulaznih i izlaznih podataka. Ovakav pristup doprinosi smanjenju grešaka u radu sistema i povećava pouzdanost komunikacije između klijentskih aplikacija i serverskog sloja.

Autentifikacija i autorizacija korisnika implementirane su korišćenjem JWT (JSON Web Token) standarda, uz dodatna prilagođavanja radi poboljšanja bezbjednosti i kontrole sesija. Ovakav pristup omogućava skalabilno i stateless upravljanje korisničkim sesijama, što je pogodno za SaaS okruženja sa većim brojem korisnika.

Za automatizovano testiranje sistema korišćen je Pytest okvir, koji omogućava pisanje jasnih i ponovljivih testova različitih nivoa, uključujući jedinične i integracione testove. Poseban fokus stavljen je na testiranje negativnih scenarija, kako bi se verifikovala ispravnost sistema u realnim uslovima rada.

Kontejnerizacija aplikacije ostvarena je korišćenjem Docker tehnologije, čime je obezbijeđena konzistentnost izvršnog okruženja između razvojnog, testnog i produkcionog sistema. Ovakav pristup olakšava distribuciju aplikacije, skaliranje i buduću nadogradnju sistema.

3 ANALIZA ZAHTJEVA I MODELOVANJE SISTEMA

U ovom poglavlju analizirani su zahtjevi razvijenog sistema i predstavljen način njegovog modelovanja sa aspekta funkcionalnosti, kvalitativnih osobina i strukture podataka. Poseban akcenat stavljen je na identifikaciju ključnih potreba koje sistem treba da zadovolji u realnom okruženju rada klinika, uključujući upravljanje korisnicima, finansijskim tokovima, bezbjednosnim mehanizmima i izolacijom podataka u multi-tenant okruženju. Pored toga, poglavlje postavlja osnovu za kasniji arhitektonski i implementacioni prikaz, jer povezuje poslovne zahtjeve sa konkretnim modelom sistema. Posebna vrijednost ovog dijela rada ogleda se u tome što se apstraktno definisani zahtjevi i modeli dalje povezuju sa načinom njihove praktične realizacije u okviru razvijenog rješenja. Na taj način se obezbjeđuje jasniji prelaz od konceptualnog nivoa sistema ka njegovim konkretnim funkcionalnostima i korisničkim scenarijima upotrebe.

U nastavku poglavlja najprije su izdvojeni funkcionalni zahtjevi, a zatim i nefunkcionalni zahtjevi koji se odnose na integritet, bezbjednost, performanse, skalabilnost i auditabilnost sistema. Nakon toga prikazan je model podataka i opisana organizacija ključnih entiteta i njihovih međusobnih relacija, čime se formira konceptualna osnova za arhitekturu i implementaciju predstavljenu u narednom poglavlju. Pored tekstualnog opisa zahtjeva i modela podataka, u završnom dijelu poglavlja dati su i reprezentativni mockup ekrani korisničkog interfejsa, koji ilustruju kako su ključne funkcionalnosti sistema organizovane iz perspektive krajnjeg korisnika. Ovi prikazi dodatno doprinose razumijevanju veze između definisanih poslovnih zahtjeva, modela podataka i konkretnih operacija koje sistem podržava u svakodnevnom radu klinike.

3.1 Funkcionalni zahtjevi

Funkcionalni zahtjevi definišu skup funkcionalnosti koje softverski sistem mora da obezbijedi kako bi ispunio potrebe korisnika i podržao predviđene poslovne procese. U kontekstu sistema za upravljanje klinikama, funkcionalni zahtjevi obuhvataju upravljanje korisnicima, finansijskim tokovima, sigurnosnim mehanizmima i administrativnim operacijama, uz jasnu podjelu odgovornosti između različitih uloga korisnika.

Sistem mora omogućiti upravljanje klinikama kao nezavisnim zakupcima unutar jedinstvene softverske platforme. Svaka klinika posjeduje sopstvene korisnike, finansijske zapise i konfiguracione parametre, pri čemu je obavezno obezbijediti potpunu logičku izolaciju

podataka između različitih zakupaca sistema. Korisnici sistema moraju imati mogućnost autentifikacije i pristupa isključivo podacima klinike kojoj pripadaju, u skladu sa dodijeljenim privilegijama.

Upravljanje korisnicima obuhvata registraciju, autentifikaciju i autorizaciju korisnika, uz podršku za različite uloge, kao što su vlasnik klinike, menadžer, doktor i recepcionar. Sistem mora omogućiti definisanje prava pristupa za svaku ulogu, kao i kontrolu izvršavanja osjetljivih operacija u skladu sa internim pravilima klinike. Posebna pažnja posvećena je podršci za rad u okruženjima sa dijeljenim terminalima, gdje je neophodno omogućiti sekundarnu autorizaciju određenih operacija bez prekida aktivne korisničke sesije.

Jedan od ključnih funkcionalnih zahtjeva odnosi se na upravljanje finansijskim obavezama pacijenata kroz planove otplate. Sistem mora omogućiti kreiranje planova sa više rata, evidentiranje djelimičnih i zakašnjelih uplata, kao i automatsku raspodjelu uplata na osnovu definisanih pravila. Sve finansijske operacije moraju biti evidentirane na način koji omogućava potpunu sljedivost i naknadnu reviziju.

Sistem mora obezbijediti centralni mehanizam za praćenje gotovinskih tokova u klinici kroz vođenje kase. Sve gotovinske uplate i isplate evidentiraju se u okviru jedinstvenog finansijskog dnevnika, uz jasnu razliku između sredstava koja pripadaju klinici i ličnih primanja zaposlenih, poput napojnica. Sistem mora omogućiti praćenje individualnih stanja napojnica po zaposlenom, kao i njihovu isplatu uz odgovarajuće odobrenje.

Dnevno zatvaranje kase predstavlja obavezan funkcionalni zahtjev sistema. Sistem mora omogućiti poređenje očekivanog stanja kase sa fizički prebrojanim iznosom, evidentiranje eventualnih odstupanja i onemogućavanje daljih finansijskih operacija nakon zaključenja dana. U zavisnosti od konfiguracije klinike, proces zatvaranja može zahtijevati dodatno odobrenje nadređenog korisnika, čime se obezbjeđuje dodatni nivo kontrole.

Sistem mora obezbijediti vođenje detaljnog revizorskog zapisa svih značajnih operacija, uključujući finansijske transakcije, promjene stanja i bezbjednosne događaje. Ovi zapisi moraju biti zaštićeni od neovlašćenih izmjena i dostupni za naknadnu analizu i provjeru integriteta sistema.

3.2 Nefunkcionalni zahtjevi

Nefunkcionalni zahtjevi definišu kvalitativne osobine sistema koje ne opisuju konkretne funkcionalnosti, već uslove pod kojima sistem mora da funkcioniše. U slučaju serverskog sistema za upravljanje klinikama, ovi zahtjevi imaju poseban značaj zbog finansijske osjetljivosti podataka, višekorisničkog okruženja i potrebe za visokim stepenom pouzdanosti i

bezbjednosti. Nefunkcionalni zahtjevi u ovom radu obuhvataju integritet podataka, bezbjednost, performanse, skalabilnost i auditabilnost sistema.

Jedan od osnovnih nefunkcionalnih zahtjeva jeste očuvanje integriteta finansijskih podataka. Sistem mora garantovati da se sve finansijske operacije izvršavaju na konzistentan i deterministički način, bez mogućnosti nastanka djelimičnih ili kontradiktornih stanja. U tom smislu, sve operacije koje mijenjaju finansijsko stanje sistema moraju biti izvršene u okviru transakcionog konteksta, uz striktno poštovanje definisanih pravila integriteta i ograničenja.

Bezbjednost sistema predstavlja ključni nefunkcionalni zahtjev, naročito u okruženju gdje više korisnika različitih uloga pristupa osjetljivim podacima. Sistem mora obezbijediti pouzdanu autentifikaciju korisnika, kontrolu pristupa zasnovanu na ulogama, kao i dodatne mehanizme autorizacije za osjetljive finansijske operacije. Posebna pažnja posvećena je radu u okruženjima sa dijeljenim terminalima, gdje je neophodno omogućiti bezbjedno izvršavanje operacija bez ugrožavanja korisničkih sesija i kontrole odgovornosti.

Auditabilnost sistema predstavlja još jedan ključni nefunkcionalni zahtjev. Sistem mora obezbijediti potpun i neizmjenjiv revizorski zapis svih značajnih operacija, posebno onih koje utiču na finansijske tokove. Revizorski zapisi moraju sadržati informacije o vremenu izvršavanja, vrsti operacije i identitetima korisnika uključenih u njen tok, uključujući korisnika koji je inicirao operaciju i korisnika koji je izvršio njenu autorizaciju. Na ovaj način obezbjeđuje se potpuna sljedivost i mogućnost naknadne analize svih aktivnosti u sistemu.

Sistem mora zadovoljiti i zahtjeve u pogledu performansi i odziva, posebno u scenarijima sa istovremenim radom većeg broja korisnika. Vrijeme obrade finansijskih operacija mora biti predvidivo i stabilno, bez obzira na broj paralelnih zahtjeva, dok mehanizmi kontrole konkurentnosti ne smiju dovesti do degradacije korisničkog iskustva. Istovremeno, sistem mora biti projektovan tako da omogući horizontalnu i vertikalnu skalabilnost u skladu sa rastom broja klinika i korisnika.

Pouzdanost sistema podrazumijeva sposobnost nastavka rada i očuvanja podataka u slučaju grešaka, prekida rada aplikacije ili infrastrukturnih problema. Sistem mora obezbijediti trajnost podataka nakon potvrde transakcija, kao i mehanizme za detekciju i evidentiranje anomalija. Ovi zahtjevi su od posebnog značaja u kontekstu finansijskih sistema, gdje gubitak ili nekonzistentnost podataka može imati ozbiljne posljedice po poslovanje klinike.

3.3 Model podataka i entitetsko-relacioni dijagram

Model podataka predstavlja centralni dio projektovanja serverskog sistema, jer definiše strukturu informacija, relacije između ključnih entiteta i pravila integriteta koja moraju biti

očuvana tokom rada aplikacije. U okviru ovog rada, model podataka je projektovan tako da podrži višekorisnički i multi-tenant rad, uz strogu izolaciju između klinika i pouzdano evidentiranje finansijskih tokova. Posebna pažnja posvećena je finansijskim entitetima, s obzirom na to da sistem mora obezbijediti tačnost transakcija, sljedivost promjena i mogućnost naknadne revizije.

U relacijskom modelu, osnovni organizacioni entitet predstavlja klinika (Clinic), koja u sistemu funkcioniše kao zakupac (tenant). Svaki zapis u sistemu koji je specifičan za jednu kliniku povezan je sa njenim identitetom, čime se obezbjeđuje logičko razdvajanje podataka između zakupaca. Na taj način se omogućava da više klinika koristi istu aplikaciju i istu šemu baze podataka, bez međusobnog uvida u podatke.

Korisnici sistema (User) pripadaju jednoj klinici i posjeduju definisanu ulogu i privilegije, čime se ostvaruje kontrola pristupa zasnovana na ulogama. Model omogućava da različiti profili korisnika obavljaju različite tipove operacija, dok se osjetljive finansijske radnje dodatno kontrolišu pravilima autorizacije i revizorskim evidencijama. U sistemu se takođe vodi evidencija pacijenata (Patient), koji predstavljaju subjekte nad kojima se formiraju finansijske obaveze i planovi otplate.

Ključni dio finansijskog modela čine planovi otplate i povezani elementi transakcija. Plan otplate (InstallmentPlan) predstavlja agregatnu finansijsku obavezu koja se sastoji od više pojedinačnih rata (Installment). Svaka rata ima definisan iznos, rok dospijeća i stanje izmirenja. Uplate (Payment) se vezuju za plan otplate, pri čemu se u sistemu primjenjuju pravila raspodjele uplate na najstarije neizmirene rate. Na ovaj način model podržava realne scenarije djelimičnih uplata, preplata i kašnjenja, bez narušavanja integriteta dugovanja.

Za praćenje gotovinskih tokova u klinici definisan je entitet kase (Cashbox), koji predstavlja centralni finansijski dnevnik kroz koji prolaze gotovinske uplate i isplate. U okviru kase se evidentira očekivano stanje i promjene nastale finansijskim operacijama, čime se omogućava precizno praćenje protoka novca. Pored osnovnog toka sredstava klinike, sistem podržava i sekundarni finansijski tok napojnica (tips), koji se vodi odvojeno od prihoda klinike. Ovaj pristup omogućava individualno praćenje napojnica po doktoru, uz kontrolu isplata i očuvanje solventnosti sekundarnog fonda, pri čemu isplata napojnica utiče na stanje kase na formalan i auditabilan način.

Mehanizam dnevnog zatvaranja (DailyClose) predstavlja ključnu tačku provjere integriteta gotovinskog poslovanja. Kroz dnevno zatvaranje sistem upoređuje očekivano stanje kase sa fizički prebrojanim iznosom, evidentira eventualna odstupanja i sprečava naknadne izmjene finansijskih evidencija za zatvoreni period. U zavisnosti od pravila klinike, proces zatvaranja može zahtijevati dodatno odobravanje nadređenog korisnika, čime se obezbjeđuje kontrola i odgovornost u osjetljivim procesima.

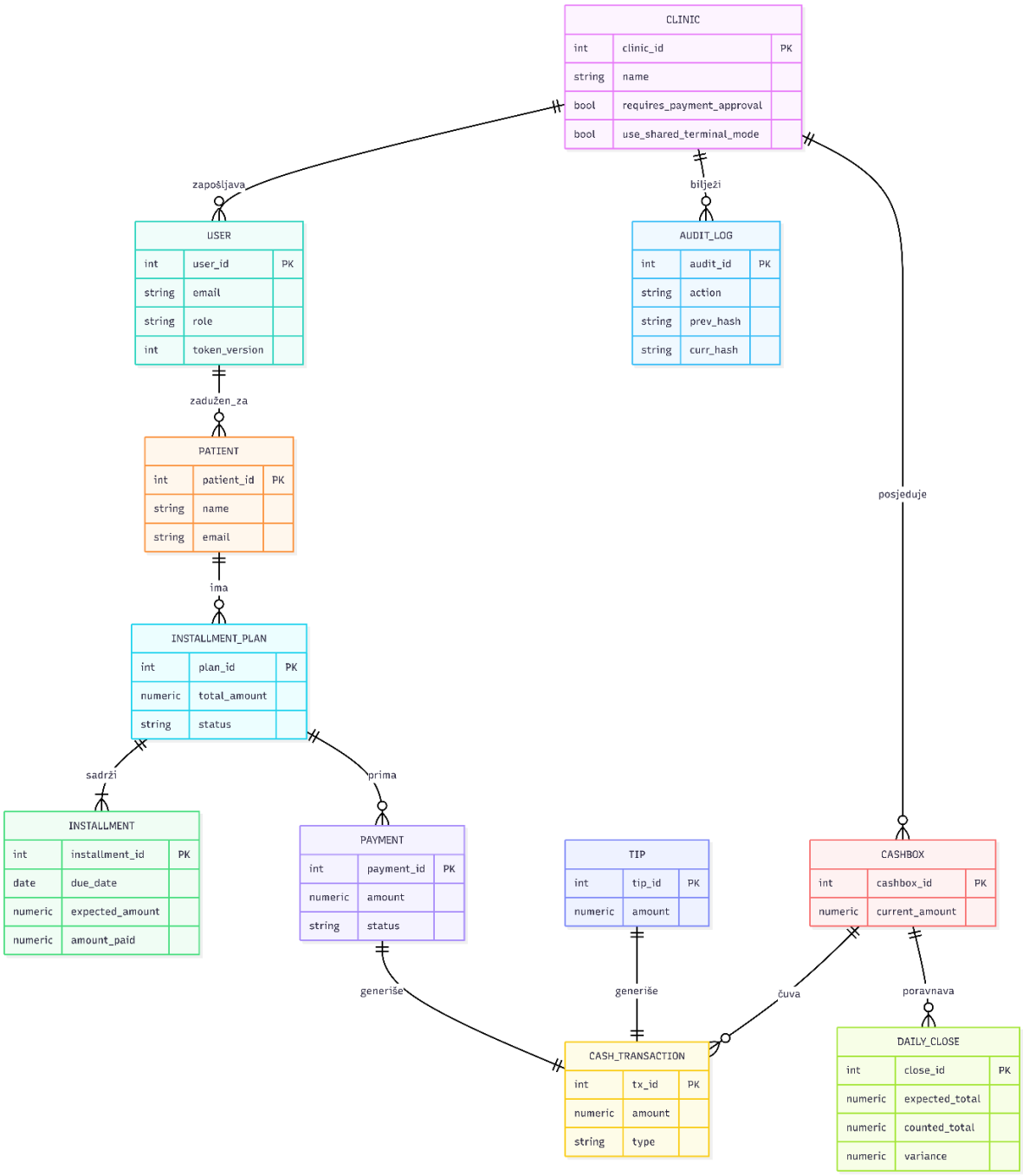
Za potrebe bezbjednosti i sljedivosti, sistem sadrži i revizorske zapise (AuditLog), koji omogućavaju rekonstrukciju događaja i detekciju neovlašćenih izmjena. Revizorski model projektovan je tako da može da zabilježi identitet korisnika koji inicira operaciju, kao i identitet korisnika koji je eventualno vrši autorizaciju, čime se obezbjeđuje potpuna odgovornost u scenarijima višeslojne autorizacije.

Na slici 3.1 prikazan je entitetsko-relacioni dijagram sistema, koji vizuelno predstavlja ključne entitete i njihove međusobne relacije. Dijagram je dat u uprošćenoj formi, sa ciljem da istakne osnovnu strukturu sistema, višekorisničku organizaciju, finansijske tokove i mehanizme revizije, bez ulaska u sve implementacione detalje fizičkog modela baze podataka.

Centralni organizacioni entitet u modelu predstavlja klinika (Clinic), koja definiše kontekst rada sistema i povezuje korisnike, finansijske evidencije i revizorske zapise u okviru jedne organizacione cjeline. Korisnici (User) su povezani sa klinikom i učestvuju u radu nad podacima pacijenata, dok pacijenti (Patient) posjeduju planove otplate (InstallmentPlan) koji se dalje sastoje od pojedinačnih rata (Installment). Uplate (Payment) se vezuju za planove otplate i predstavljaju osnovu finansijskog toka koji se dalje reflektuje kroz gotovinske transakcije (CashTransaction).

Poseban dio modela odnosi se na upravljanje gotovinom i kontrolu finansijskih stanja. Entitet kasa (Cashbox) čuva trenutno stanje gotovine i povezan je sa gotovinskim transakcijama, dok entitet dnevnog zatvaranja (DailyClose) omogućava poređenje očekivanog i fizički prebrojanog stanja kase. Na taj način model podržava svakodnevnu kontrolu finansijskog poslovanja i evidentiranje eventualnih odstupanja.

Pored osnovnih finansijskih tokova, model obuhvata i entitet napojnice (Tip), koji je odvojen od regularnih uplata i generiše odgovarajuće gotovinske transakcije, kao i entitet audit loga (AuditLog), čime se obezbjeđuje revizorski trag svih značajnih promjena u sistemu. Ovakva struktura modela omogućava jasnu organizaciju poslovne logike i predstavlja osnovu za arhitekturu i implementaciju opisanu u narednim poglavljima.

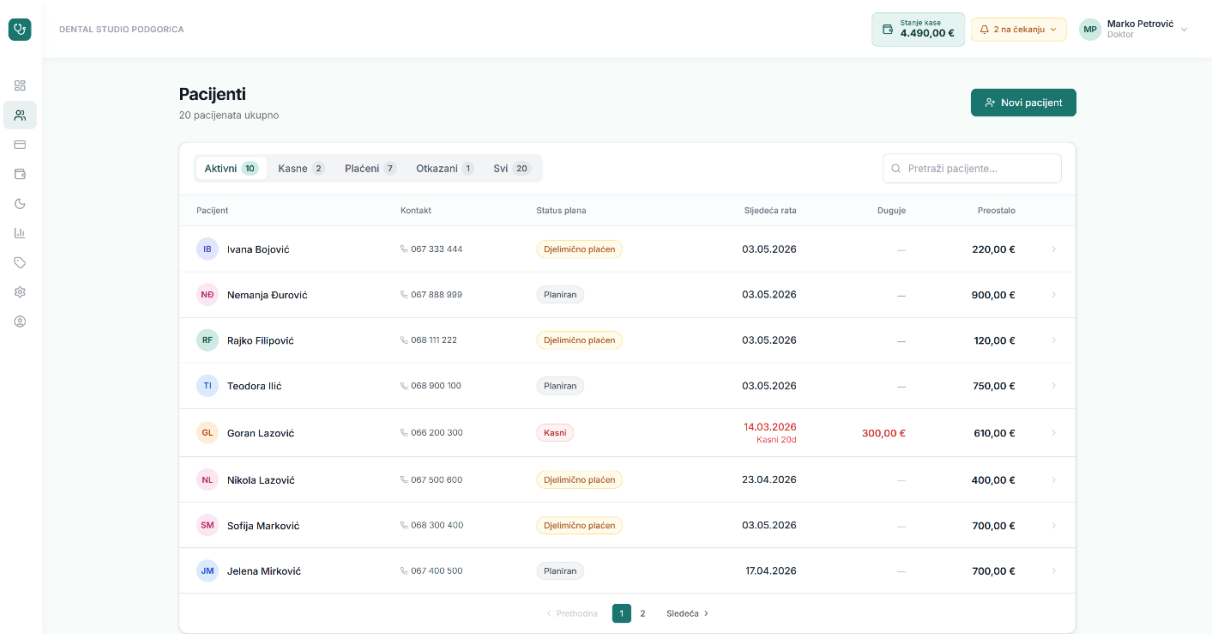


Slika 3.1: Entitetsko-relacioni dijagram sistema za upravljanje klinikama

3.4 Mockup ekrani korisničkog interfejsa

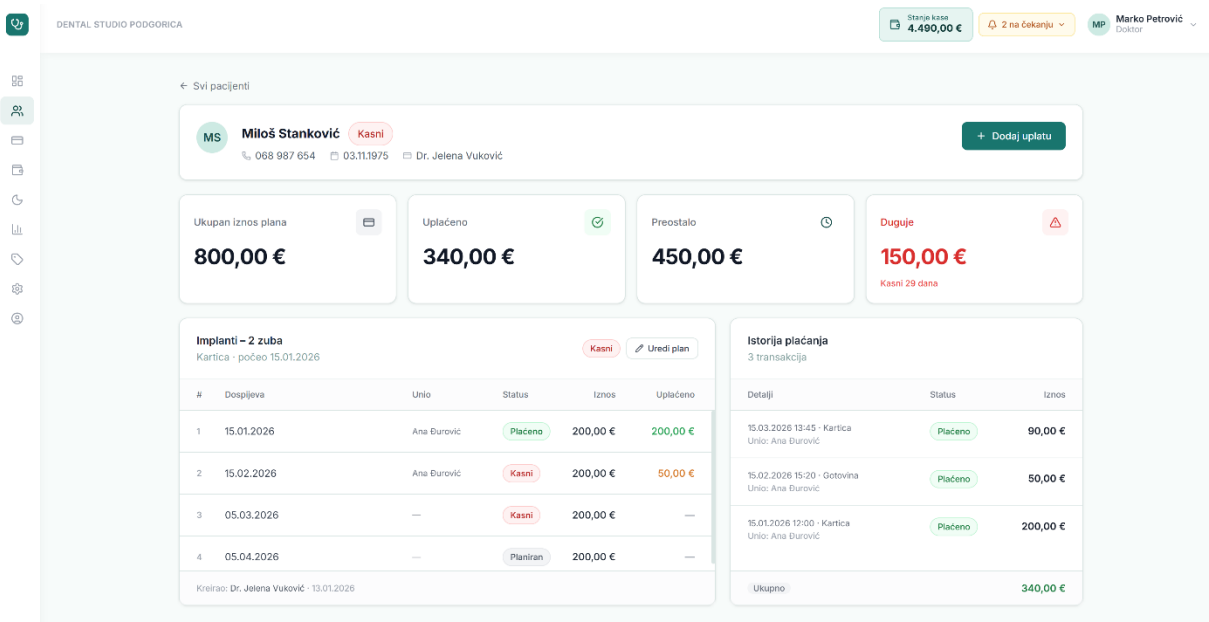
U cilju potpunijeg prikaza projektovanog sistema, pored modela podataka i funkcionalnih zahtjeva korisno je prikazati i reprezentativne ekrane korisničkog interfejsa. Iako je fokus ovog rada primarno na serverskoj arhitekturi, poslovnoj logici i obradi finansijskih tokova, korisnički interfejs predstavlja važan sloj kroz koji se realizuju svakodnevne operacije u radu klinike. Prikazani ekrani ilustruju način na koji su ključne funkcionalnosti sistema organizovane iz perspektive krajnjeg korisnika, uključujući pregled pacijenata, upravljanje planovima otplate, evidentiranje gotovinskih transakcija i proces dnevnog zatvaranja kase. Na taj način se uspostavlja jasnija veza između definisanih zahtjeva sistema i njihove konkretne primjene u praksi.

Na slici 3.2 prikazan je ekran za pregled pacijenata, koji predstavlja jednu od centralnih ulaznih tačaka za svakodnevni rad u sistemu. Ekran omogućava tabelarni prikaz aktivnih i arhiviranih pacijenata, uz osnovne informacije kao što su kontakt podaci, status plana otplate, naredna rata i preostali dug. Pored toga, interfejs podržava pretragu i filtriranje po statusima, čime se korisnicima omogućava brza identifikacija pacijenata koji kasne sa uplatom ili imaju aktivne finansijske obaveze. Ovakva organizacija interfejsa direktno podržava funkcionalne zahtjeve vezane za pregled i upravljanje pacijentima u okviru jedne klinike.



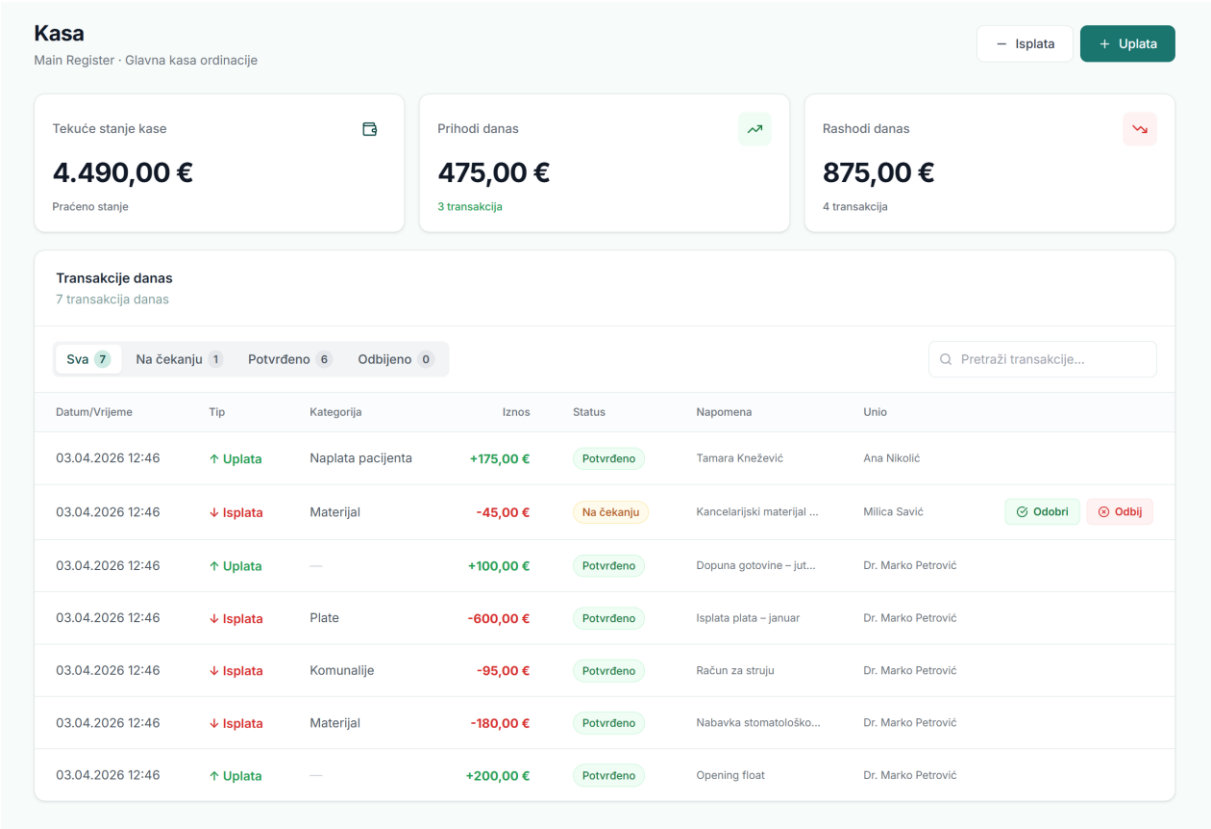
Slika 3.2: Ekran za pregled pacijenata

Na slici 3.3 prikazan je ekran detalja pacijenta, koji objedinjeno prikazuje najvažnije informacije o njegovom planu otplate i istoriji finansijskih transakcija. U okviru ovog prikaza mogu se vidjeti ukupan iznos plana, do tada uplaćeni iznos, preostali dug, status dospjelih rata i istorija izvršenih uplata. Posebna vrijednost ovog ekrana ogleda se u tome što povezuje podatke o pacijentu, planu otplate, pojedinačnim ratama i izvršenim plaćanjima u jedinstvenu radnu cjelinu. Na taj način se jasno ilustruje kako sistem podržava realne scenarije djelimičnih uplata, kašnjenja i praćenja finansijskog statusa pacijenta kroz vrijeme.



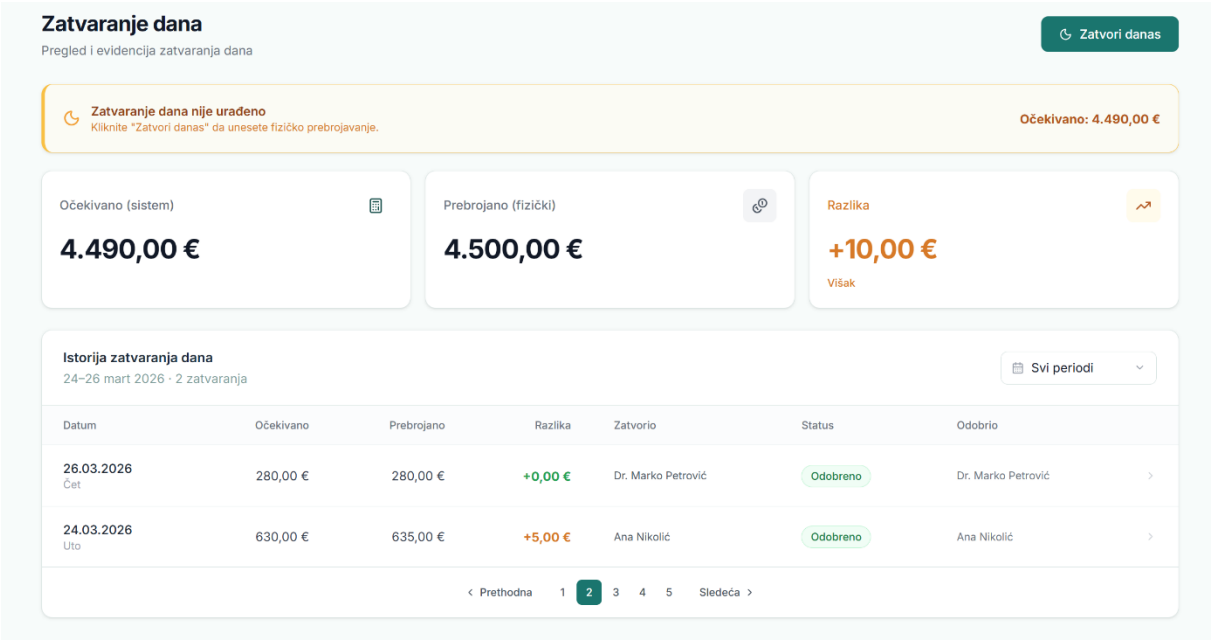
Slika 3.3: Ekran detalja pacijenta sa planom otplate i istorijom plaćanja

Na slici 3.4 prikazan je ekran kase, namijenjen evidenciji i pregledu gotovinskih tokova u okviru klinike. Ekran prikazuje trenutno stanje kase, dnevni pregled prihoda i rashoda, kao i listu pojedinačnih transakcija sa odgovarajućim statusima i kategorijama. Kroz ovakav prikaz korisniku je omogućeno da na jednom mjestu prati sve promjene gotovinskog stanja, uključujući uplate pacijenata, interne dopune kase i različite vrste isplata. Ovaj ekran direktno odražava arhitektonsko i poslovno rješenje sistema, prema kojem se stanje kase ne mijenja proizvoljno, već isključivo kroz formalno evidentirane transakcije.



Slika 3.4: Ekran kase sa pregledom gotovinskih transakcija

Na slici 3.5 prikazan je ekran za dnevno zatvaranje kase, koji predstavlja jednu od ključnih kontrolnih funkcionalnosti sistema. Ovaj prikaz omogućava poređenje očekivanog stanja kase sa fizički prebrojanim iznosom, evidentiranje eventualne razlike i pregled istorije prethodnih zatvaranja. Na taj način korisniku je omogućen jasan uvid u finansijsko stanje na kraju radnog dana, kao i praćenje odstupanja i statusa odobravanja zatvaranja. Prikaz ovog ekrana je od posebnog značaja jer ilustruje praktičnu realizaciju mehanizma dnevnog zatvaranja, koji ima centralnu ulogu u očuvanju integriteta gotovinskog poslovanja i revizorske pouzdanosti sistema.



Slika 3.5: Ekran dnevnog zatvaranja kase sa istorijom zatvaranja

4 ARHITEKTURA I IMPLEMENTACIJA SISTEMA

U ovom poglavlju prikazana je arhitektura razvijenog serverskog sistema za upravljanje klinikama, kao i ključni aspekti njegove implementacije. Poseban fokus stavljen je na organizaciju softverskih komponenti, razdvajanje odgovornosti između slojeva sistema i način na koji arhitektonska rješenja podržavaju zahtjeve integriteta finansijskih podataka, konkurentnog pristupa i bezbjednosti u multi-tenant okruženju. Opisana arhitektura predstavlja osnovu za pouzdano izvršavanje poslovne logike i efikasnu obradu finansijskih transakcija u uslovima paralelnog rada više korisnika.

U nastavku poglavlja dat je opšti arhitektonski pregled sistema, nakon čega su detaljno analizirani ključni implementacioni mehanizmi, uključujući sloj poslovne logike, upravljanje transakcijama, obradu finansijskih tokova, dnevno zatvaranje kase i bezbjednosne mehanizme. Pored tekstualnog opisa implementacije, prikazani su i UML dijagram klasa podsistema dnevnog zatvaranja kase, kao i sekvencijski dijagram procesa njegovog kreiranja, čime su dodatno ilustrovani strukturni i dinamički aspekti jedne od najvažnijih finansijskih operacija u sistemu.

4.1 Opšti arhitektonski pregled sistema

Razvijeni serverski sistem projektovan je u skladu sa principima višeslojne softverske arhitekture, sa ciljem jasnog razdvajanja odgovornosti, povećanja održivosti koda i smanjenja međuzavisnosti između komponenti. U radu je primijenjena troslojna arhitektura, koja se sastoji od prezentacionog sloja (API), sloja poslovne logike i sloja pristupa podacima. Ovakav pristup omogućava kontrolisanu obradu zahtjeva, dosljednu primjenu poslovnih pravila i pouzdanu interakciju sa bazom podataka.

Prezentacioni sloj sistema realizovan je kroz API rute, koje su organizovane korišćenjem Flask *blueprint* mehanizma. Ovaj sloj je zadužen isključivo za obradu HTTP zahtjeva i odgovora, validaciju ulaznih podataka i prosleđivanje kontrolisanih poziva ka nižim slojevima sistema. API sloj ne sadrži poslovnu logiku, čime se obezbjeđuje njegova jednostavnost i lakša izmjenjivost, kao i mogućnost kasnije integracije različitih klijentskih aplikacija.

Sloj poslovne logike implementiran je kroz servisne komponente, koje predstavljaju centralno mjesto za realizaciju funkcionalnosti sistema. Servisi enkapsuliraju pravila obrade finansijskih transakcija, upravljanje planovima otplate, kontrolu konkurentnog pristupa i sprovođenje bezbjednosnih ograničenja. Na ovaj način se obezbjeđuje da su svi ključni procesi implementirani na jednom mjestu, nezavisno od načina na koji su inicirani putem API-ja.

Sloj pristupa podacima realizovan je kroz poseban modul za rad sa bazom podataka, koji sadrži repozitorijumske komponente zadužene za izvršavanje upita i manipulaciju perzistentnim podacima. Ovaj sloj predstavlja jedinu tačku komunikacije sa bazom podataka i omogućava izolaciju SQL logike od poslovnih pravila sistema. Takva organizacija doprinosi većoj testabilnosti i smanjuje rizik od nekonzistentnih pristupa podacima.

Dodatno, za validaciju i serijalizaciju podataka korišćeni su formalni šematski modeli, čime se obezbjeđuje striktna kontrola strukture ulaznih i izlaznih podataka. Ovaj mehanizam doprinosi pouzdanosti sistema i smanjuje mogućnost pojave grešaka izazvanih neispravnim ili nepotpunim podacima.

Opisana arhitektura omogućava jasan tok izvršavanja zahtjeva, od prijema API poziva, preko obrade poslovne logike, do perzistencije podataka, uz očuvanje modularnosti i skalabilnosti sistema. Takva organizacija predstavlja stabilnu osnovu za dalji razvoj i proširenje funkcionalnosti aplikacije.

4.2 Sloj servisa i upravljanje poslovnom logikom

Sloj poslovne logike predstavlja centralni dio implementacije sistema, jer u njemu se realizuju sva ključna pravila i procesi vezani za upravljanje finansijskim tokovima, kontrolu konkurentnog pristupa i sprovođenje bezbjednosnih ograničenja. Ovaj sloj je implementiran kroz skup servisnih komponenti, koje djeluju kao posrednik između API sloja i sloja pristupa podacima.

Servisne komponente su projektovane tako da enkapsuliraju kompletnu poslovnu logiku sistema, bez direktne zavisnosti od HTTP konteksta ili detalja perzistencije. Na taj način je obezbijeđeno da se poslovna pravila primjenjuju konzistentno, nezavisno od izvora zahtjeva, kao i da se isti servisi mogu koristiti u različitim kontekstima, uključujući testiranje i potencijalne buduće ekstenzije sistema.

Posebna pažnja u implementaciji poslovne logike posvećena je finansijskim operacijama, s obzirom na njihovu osjetljivost i potrebu za očuvanjem integriteta podataka. Finansijski procesi, poput obrade uplata, raspodjele sredstava po ratama i evidentiranja gotovinskih transakcija, realizovani su kroz jasno definisane servisne metode koje sprovode strogo kontrolisane tranzicije stanja. Na taj način se sprječava pojava nekonzistentnih ili djelimično izvršenih operacija.

Jedan od ključnih elemenata poslovne logike jeste mehanizam obrade uplata u okviru planova otplate. Prilikom evidentiranja uplate, sistem automatski raspoređuje uplaćeni iznos na najstarije neizmirene rate, uz podršku djelimičnih uplata i preplata. Ovaj algoritamski

pristup omogućava da se realni scenariji naplate obrađuju korektno i deterministički, bez potrebe za ručnom intervencijom operatera.

Kako bi se obezbijedila sigurnost u uslovima paralelnog rada više korisnika, poslovna logika uključuje mehanizme kontrole konkurentnog pristupa. Kritične finansijske operacije izvršavaju se unutar transakcionih cjelina, pri čemu se koriste mehanizmi zaključavanja na nivou zapisa baze podataka. Time se garantuje da istovremeni zahtjevi ne mogu dovesti do narušavanja finansijskog stanja ili dvostruke obrade iste obaveze.

Sistem dodatno uvodi razdvajanje primarnih i sekundarnih finansijskih tokova. Pored glavnog toka prihoda klinike, poslovna logika podržava i nezavisan tok napojnica, koji se vodi odvojeno od osnovnog finansijskog bilansa. Ovaj sekundarni tok podataka omogućava individualno praćenje iznosa dostupnih za isplatu po zaposlenom, uz stroga pravila solventnosti i kontrolu uticaja isplata na stanje kase.

Implementacija poslovne logike obuhvata i mehanizme za dnevno finansijsko zatvaranje, kojim se vrši formalna provjera stanja kase. Tokom ovog procesa, sistem upoređuje očekivano stanje sa fizički prebrojanim iznosom i evidentira eventualna odstupanja kroz posebne finansijske zapise. Nakon zatvaranja, poslovna logika onemogućava naknadne izmjene finansijskih podataka za zatvoreni period, čime se obezbjeđuje vremenska konzistentnost i revizorska pouzdanost sistema.

Na ovaj način, sloj poslovne logike predstavlja temelj sistema, jer objedinjuje finansijska pravila, bezbjednosne kontrole i mehanizme konkurentnosti u jedinstvenu i konzistentnu cjelinu, prilagođenu realnim uslovima rada u medicinskim ustanovama.

4.3 Upravljanje transakcijama i konkurentni pristup

U višekorisničkim sistemima koji obrađuju finansijske podatke, kontrola konkurentnog pristupa predstavlja jedan od ključnih izazova. Istovremeni rad više operatera nad istim finansijskim resursima može dovesti do nekonzistentnih stanja, duplih zaduženja ili gubitka podataka ukoliko se ne primijene odgovarajući mehanizmi sinhronizacije. Iz tog razloga, u okviru ovog sistema poseban akcenat stavljen je na pravilno upravljanje transakcijama i obezbjeđivanje konkurentne bezbjednosti.

Sistem se oslanja na transakcioni model relacione baze podataka, uz dosljedno poštovanje ACID svojstava (atomarnost, konzistentnost, izolacija i trajnost). Sve finansijski osjetljive operacije izvršavaju se unutar jasno definisanih transakcionih cjelina, čime se obezbjeđuje da se složene operacije, poput obrade uplata ili zatvaranja kase, izvrše u potpunosti ili da se u slučaju greške stanje sistema vrati u prethodno konzistentno stanje.

Posebno rizične tačke u sistemu predstavljaju operacije koje uključuju raspodjelu sredstava, kao što su uplate po planovima otplate i ažuriranje stanja kase. Kako bi se spriječile situacije u kojima više korisnika istovremeno pokušava da izmijeni iste zapise, primijenjeni su mehanizmi pesimističkog zaključavanja na nivou redova baze podataka. Ovim pristupom se obezbjeđuje da, u trenutku obrade finansijske operacije, relevantni zapisi budu zaključani za ostale transakcije sve dok se tekuća operacija ne završi.

Primjena zaključavanja na nivou redova omogućava sistemu da bezbjedno obrađuje paralelne zahtjeve, kao što su istovremene uplate ili pokušaji zatvaranja kase, bez rizika od tzv. race condition scenarija. Na ovaj način se eliminiše mogućnost da dvije transakcije istovremeno promijene isto finansijsko stanje, što je posebno važno u okruženjima sa zajedničkim terminalima i intenzivnim radom na recepciji.

U okviru obrade uplata, konkurentni mehanizmi su kombinovani sa strogo kontrolisanim tranzicijama stanja. Svaka uplata prolazi kroz definisani životni ciklus, pri čemu se promjene statusa dozvoljavaju isključivo u skladu sa poslovnim pravilima sistema. Time se dodatno smanjuje mogućnost nekonzistentnih stanja izazvanih paralelnim ili neautorizovanim operacijama.

Sličan princip primijenjen je i kod procesa dnevnog zatvaranja kase. Tokom zatvaranja, sistem zaključava relevantne finansijske zapise kako bi spriječio naknadne izmjene u periodu koji se zatvara. U slučaju da pravila klinike zahtijevaju višeslojno odobravanje, proces zatvaranja ostaje u privremenom stanju sve dok ne budu ispunjeni svi uslovi, čime se obezbjeđuje potpuna kontrola nad završnim finansijskim izvještajem.

Važno je napomenuti da su mehanizmi konkurentnog pristupa projektovani tako da budu transparentni za korisnike sistema, ali strogi u pogledu očuvanja integriteta podataka. Na ovaj način sistem omogućava efikasan rad u realnom okruženju, bez ugrožavanja tačnosti finansijskih informacija i revizorske pouzdanosti.

4.4 Upravljanje finansijskim tokovima i dnevno zatvaranje

Upravljanje finansijskim tokovima predstavlja jednu od centralnih funkcionalnosti razvijenog sistema, s obzirom na to da greške u ovom segmentu mogu imati direktne poslovne i pravne posljedice. Sistem je projektovan tako da omogući precizno praćenje svih novčanih tokova u klinici, uključujući uplate pacijenata, gotovinske transakcije, sekundarne tokove napojnica i proces dnevnog finansijskog zatvaranja.

Osnovni mehanizam za praćenje gotovine u sistemu predstavlja entitet kase (Cashbox), koji funkcioniše kao centralni finansijski dnevnik klinike. Svaka klinika može imati jednu ili više

kasa, pri čemu se sve gotovinske operacije evidentiraju kroz formalizovane transakcije. Stanje kase se ne mijenja direktnim ažuriranjem vrijednosti, već isključivo kroz jasno definisane finansijske događaje, čime se obezbjeđuje potpuna sljedivost promjena.

Uplate pacijenata se u sistemu obrađuju kroz planove otplate, koji omogućavaju realne scenarije djelimičnih uplata, preplata i kašnjenja. Prilikom evidentiranja uplate, sistem automatski raspoređuje uplaćeni iznos na najstarije neizmirene rate prema unaprijed definisanim pravilima. Ovaj proces obezbjeđuje da finansijsko stanje pacijenta uvijek ostane konzistentno, bez obzira na redoslijed i iznos pojedinačnih uplata.

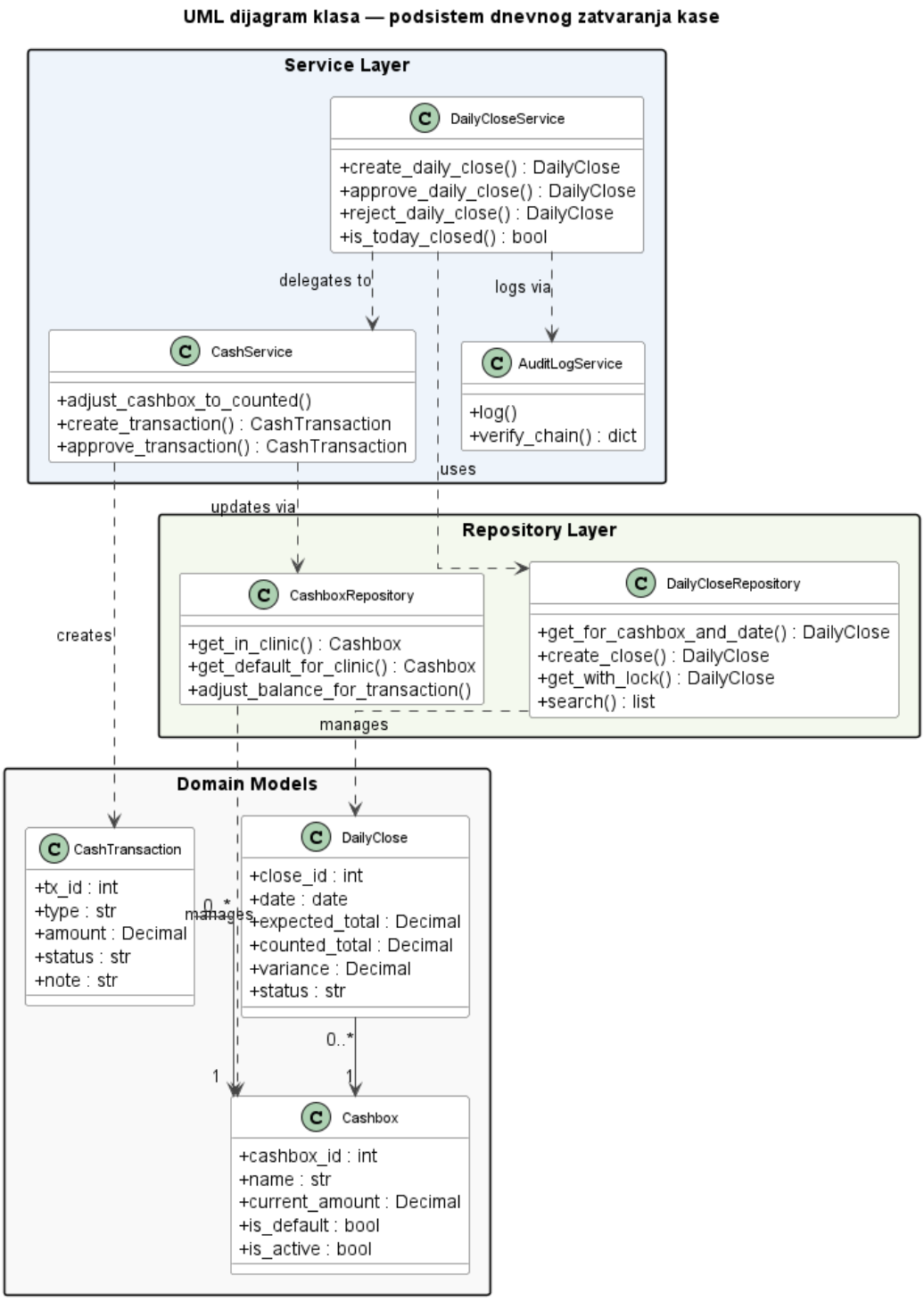
Pored osnovnog toka sredstava klinike, sistem podržava i sekundarni finansijski tok napojnica. Napojnice se vode kao odvojeni fond vezan za pojedinačne doktore i ne miješaju se direktno sa prihodom klinike. Sistem kontinuirano prati raspoloživi iznos napojnica po doktoru i onemogućava isplatu iznosa koji premašuje stvarno prikupljena sredstva. Prilikom isplate napojnica, odgovarajući iznos se formalno evidentira kao transakcija koja utiče na stanje kase, čime se očuvava finansijska konzistentnost sistema.

Posebno značajan mehanizam sistema predstavlja proces dnevnog zatvaranja kase (Daily Close). Ovaj proces ima za cilj da izvrši formalnu provjeru usklađenosti između očekivanog stanja kase, koje proizilazi iz evidentiranih transakcija, i fizički prebrojanog iznosa gotovine. Tokom zatvaranja, sistem izračunava eventualno odstupanje između ove dvije vrijednosti i evidentira ga kao sastavni dio finansijskog izvještaja.

U zavisnosti od konfiguracije klinike, proces dnevnog zatvaranja može zahtijevati dodatni nivo odobravanja od strane nadređenog korisnika. U takvim slučajevima, zatvaranje ostaje u privremenom stanju dok se ne izvrši potrebna autorizacija, čime se sprečava zaključavanje kase u prisustvu neriješenih ili spornih finansijskih stavki. Nakon uspješnog zatvaranja, sistem onemogućava naknadno dodavanje ili izmjenu transakcija za zatvoreni period, čime se obezbjeđuje integritet istorijskih finansijskih podataka.

Kombinacijom ovih mehanizama, razvijeni sistem omogućava pouzdano upravljanje finansijskim tokovima u realnom radnom okruženju, uz jasnu razdvojenost različitih tipova sredstava i strogu kontrolu kritičnih finansijskih procesa.

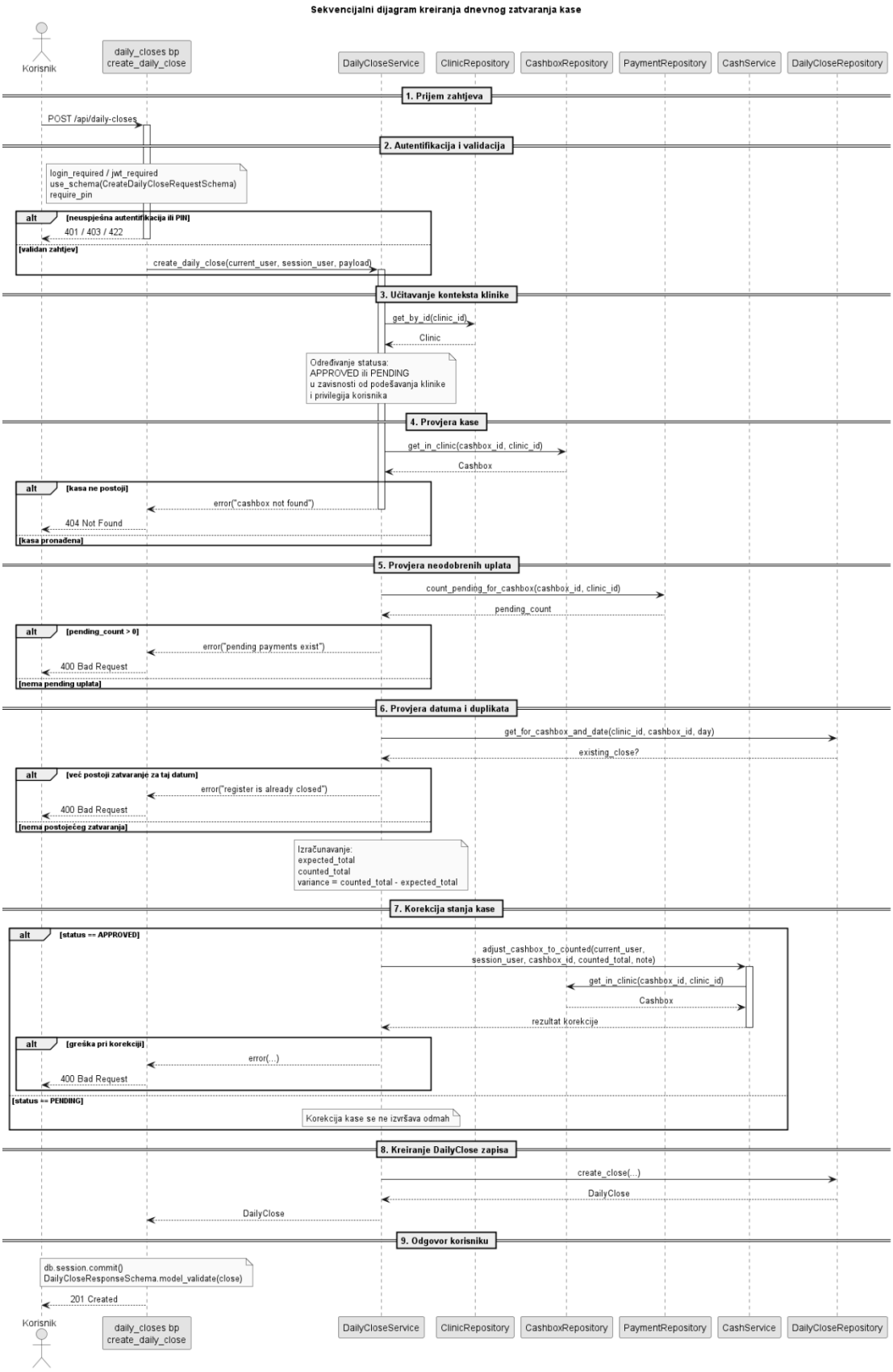
S obzirom na to da proces dnevnog zatvaranja kase obuhvata više međusobno povezanih komponenti i jasno definisane odgovornosti unutar arhitekture sistema, njegovu implementaciju je korisno prikazati i kroz UML dijagram klasa. U nastavku je dat klasni dijagram koji prikazuje ključne domenske entitete, servisne komponente i repozitorijumske zavisnosti uključene u realizaciju ovog procesa. Na taj način se, pored dinamičkog prikaza toka izvršavanja, dobija i strukturni pogled na podsistem dnevnog zatvaranja, čime se jasnije uočava raspodjela poslovne logike, način pristupa podacima i povezanost finansijskih elemenata koji učestvuju u obradi i kontroli dnevnog zatvaranja kase.



Slika 4.1: UML dijagram klasa podsistema dnevnog zatvaranja kase

S obzirom na to da proces dnevnog zatvaranja kase obuhvata više međusobno povezanih koraka i poslovnih provjera, njegovo ponašanje je korisno prikazati i kroz dijagram interakcije. U nastavku je dat sekvencijski dijagram koji prikazuje tok kreiranja dnevnog zatvaranja, počev od prijema zahtjeva i njegove validacije, preko učitavanja konteksta klinike i provjere relevantnih finansijskih uslova, pa sve do eventualne korekcije stanja kase i kreiranja

odgovarajućeg zapisa u bazi podataka. Dijagram posebno naglašava grane izvršavanja u slučaju greške ili neispunjenih uslova, čime se jasno prikazuju mehanizmi kojima sistem štiti konzistentnost finansijskih podataka i sprječava nedozvoljene ili kontradiktorne operacije.



Slika 4.2: Sekvencijski dijagram kreiranja dnevnog zatvaranja kase

4.5 Revizorski trag i bezbjednosni mehanizmi

U finansijskim informacionim sistemima, bezbjednost i mogućnost naknadne revizije predstavljaju osnovne preduslove za pouzdan rad. Pored ispravnosti same poslovne logike, neophodno je obezbijediti mehanizme koji omogućavaju detekciju neovlašćenih izmjena, jasnu atribuciju odgovornosti i trenutno reagovanje u slučaju kompromitacije korisničkih naloga. U okviru razvijenog sistema, ovi zahtjevi su adresirani kroz kombinaciju revizorskog traga, višeslojne autorizacije i naprednog upravljanja autentifikacionim sesijama.

Centralni element revizije predstavlja entitet revizorskog zapisa (AuditLog), koji se koristi za evidentiranje svih osjetljivih operacija u sistemu, posebno onih koje utiču na finansijske podatke. Svaki revizorski zapis sadrži informacije o izvršenoj akciji, zahvaćenom entitetu, vremenu izvršenja i identitetu korisnika koji je inicirao operaciju. Dodatno, sistem omogućava da se zabilježi i identitet korisnika koji je izvršio autorizaciju operacije, u slučajevima kada je primijenjen mehanizam sekundarne potvrde.

Kako bi se obezbijedila otpornost na naknadne manipulacije revizorskih podataka, audit log je implementiran kao kriptografski povezan lanac zapisa. Svaki novi zapis sadrži kriptografski sažetak prethodnog zapisa, čime se formira neprekidna hash-veza između događaja. Na ovaj način, svaka izmjena ili brisanje postojećeg zapisa može se detektovati narušavanjem kontinuiteta lanca, što omogućava pouzdanu forenzičku analizu u slučaju sumnje na zloupotrebu.

Bezbjednosna arhitektura sistema zasnovana je na kontroli pristupa prema ulogama korisnika (Role-Based Access Control – RBAC). Korisnicima su dodijeljene jasno definisane uloge i privilegije koje određuju koje operacije mogu izvršavati. Posebna pažnja posvećena je finansijskim operacijama, koje mogu zahtijevati dodatni nivo odobravanja u zavisnosti od konfiguracije klinike i privilegija korisnika.

U realnim radnim uslovima, medicinske ustanove često koriste zajedničke radne stanice, što otežava primjenu klasičnih mehanizama prijave i odjave korisnika. Kako bi se ovaj problem riješio, sistem uvodi mehanizam sekundarne autorizacije zasnovan na ličnom identifikacionom broju (PIN). U situacijama kada je potrebno brzo odobravanje osjetljive operacije, ovlašćeni korisnik može izvršiti autorizaciju unosom svog PIN-a, bez prekidanja aktivne sesije drugog korisnika. Ovakve akcije se u revizorskom zapisu jasno razdvajaju na korisnika koji je inicirao operaciju i korisnika koji je izvršio autorizaciju, čime se zadržava puna odgovornost svih učesnika.

Dodatni sloj bezbjednosti uveden je kroz mehanizam verzionisanja autentifikacionih tokena. Standardni JWT mehanizmi ne omogućavaju trenutno poništavanje već izdatih tokena, što predstavlja bezbjednosni rizik u slučaju kompromitacije naloga. U ovom sistemu, svaki korisnik posjeduje verziju tokena koja se uključuje u sadržaj autentifikacionog tokena. Prilikom svake provjere pristupa, sistem upoređuje verziju tokena sa trenutnom vrijednošću u bazi podataka. Promjena lozinke ili bezbjednosnih parametara automatski invalidira sve prethodno izdate tokene, bez potrebe za dodatnim skladištenjem stanja sesija.

Kombinacijom kriptografski zaštićenog revizorskog traga, višeslojne autorizacije i kontrole sesija, razvijeni sistem obezbjeđuje visok nivo bezbjednosti i transparentnosti. Ovi mehanizmi omogućavaju ne samo preventivnu zaštitu od zloupotreba, već i pouzdanu rekonstrukciju događaja, što je od posebnog značaja u finansijskom okruženju medicinskih ustanova.

5 VERIFIKACIJA I TESTIRANJE SISTEMA

U ovom poglavlju prikazan je pristup verifikaciji i testiranju razvijenog sistema, sa ciljem provjere ispravnosti implementirane poslovne logike, bezbjednosnih mehanizama i konzistentnosti obrade finansijskih transakcija. Poseban fokus stavljen je na testiranje scenarija koji su od presudnog značaja za pouzdan rad sistema, uključujući konkurentni pristup zajedničkim resursima, obradu finansijskih tokova i kontrolu osjetljivih operacija. Na taj način se procjenjuje u kojoj mjeri implementirano rješenje odgovara definisanim zahtjevima i obezbjeđuje stabilno ponašanje u uslovima realnog korišćenja.

U nastavku poglavlja najprije je opisana strategija testiranja, a zatim su predstavljeni jedinični i integracioni testovi korišćeni za provjeru pojedinačnih komponenti i njihovog međusobnog rada. Nakon toga razmotreno je testiranje bezbjednosnih mehanizama, kao i ograničenja testnog okruženja u odnosu na produkcionu bazu podataka. Na kraju su sumirani rezultati testiranja, uključujući i analizu pokrivenosti koda testovima, koja pruža dodatni kvantitativni pokazatelj obuhvata verifikacije sistema.

5.1 Strategija testiranja

Verifikacija ispravnosti sistema sprovedena je kroz kombinaciju jediničnih i integracionih testova, sa posebnim fokusom na finansijsku tačnost, bezbjednosne mehanizme i ponašanje sistema u graničnim i nepovoljnim scenarijima. Umjesto testiranja isključivo očekivanih tokova izvršavanja, strategija testiranja je usmjerena na provjeru integriteta sistema u situacijama koje u praksi predstavljaju najveći rizik, kao što su djelimične uplate, paralelne operacije, pokušaji neovlašćenih akcija i narušavanje finansijskih pravila.

Testovi su organizovani po funkcionalnim cjelinama sistema i pokrivaju infrastrukturne komponente, finansijsku logiku, bezbjednosne mehanizme i API sloj. Na ovaj način je omogućeno da se pojedinačni dijelovi sistema testiraju izolovano, ali i da se potvrdi njihova međusobna saradnja u realističnim scenarijima korišćenja.

5.2 Jedinični i integracioni testovi

Testni skup obuhvata ukupno dvanaest testnih modula, kao i dodatni modul za testiranje autentifikacionih API ruta, pri čemu je izvršeno ukupno sedamdeset četiri testna slučaja. Testovi su realizovani korišćenjem PyTest okvira i izvršavaju se nad testnom instancom sistema sa izolovanom bazom podataka.

Posebna pažnja posvećena je testiranju finansijskog jezgra sistema. Testni slučajevi obuhvataju scenarije instantnih uplata, uplata koje zahtijevaju odobravanje, raspodjelu sredstava po principu vodopada, tretman preplata kao napojnica, kao i kompleksne kombinacije sekvencijalnih uplata i isplata. Na ovaj način potvrđeno je da sistem dosljedno održava tačnost dugovanja i stanja finansijskih fondova.

Integracioni testovi dodatno verifikuju pravilno ponašanje sistema u procesima dnevnog zatvaranja kase. Testirani su slučajevi potpunog poklapanja iznosa, odstupanja između očekivanog i prebrojanog stanja, blokade zatvaranja usljed neriješenih uplata, kao i ponašanje sistema u slučaju pokušaja ponovnog ili retroaktivnog zatvaranja. Ovi testovi potvrđuju da sistem ispravno sprovodi poslovna pravila i sprječava narušavanje integriteta finansijskih podataka.

5.3 Testiranje bezbjednosnih mehanizama

Bezbjednosni aspekti sistema testirani su kroz posebne testne scenarije koji obuhvataju autentifikaciju, autorizaciju i revizorski trag. Testirani su procesi registracije i prijave korisnika, odbijanje pristupa neaktivnim nalozima, kao i mehanizam poništavanja autentifikacionih tokena nakon promjene lozinke.

Posebni testovi posvećeni su provjeri revizorskog zapisa, uključujući ispravno formiranje hash-lanca i detekciju neovlašćenih izmjena zapisa. Time je potvrđeno da sistem može pouzdano detektovati pokušaje manipulacije revizorskim podacima.

Testiran je i mehanizam sekundarne autorizacije zasnovan na PIN-u, koji omogućava odobravanje osjetljivih operacija u okruženjima sa zajedničkim terminalima. Testni scenariji potvrđuju da se identitet korisnika koji inicira operaciju i korisnika koji je autorizuje pravilno evidentira, čime se obezbjeđuje potpuna sljedivost odgovornosti.

5.4 Ograničenja testnog okruženja i razlika između testne i produkcione baze

Testiranje sistema izvršeno je korišćenjem SQLite baze podataka, koja je odabrana zbog jednostavnosti konfiguracije i brzine izvršavanja testova. Međutim, SQLite ne pruža punu podršku za napredne mehanizme zaključavanja i paralelno izvršavanje transakcija, što predstavlja ograničenje u kontekstu testiranja konkurentnog pristupa.

Zbog navedenih ograničenja, stres testovi koji simuliraju veliki broj istovremenih finansijskih operacija nisu mogli biti pouzdano sprovedeni u testnom okruženju. Iako je odgovarajući testni scenario bio projektovan, njegovo izvršavanje nad SQLite bazom ne bi dalo relevantne rezultate u poređenju sa produkcionim okruženjem zasnovanim na PostgreSQL bazi podataka.

U produkcionom okruženju, sistem koristi mehanizme zaključavanja na nivou redova i transakcioni model PostgreSQL baze, čime se obezbjeđuje bezbjedno izvršavanje konkurentnih operacija. Razlika između testnog i produkcionog okruženja jasno je identifikovana i predstavlja svjesno dizajnersko ograničenje testne faze, a ne nedostatak same arhitekture sistema.

5.5 Rezultati testiranja

Rezultati izvršenih testova potvrđuju da sistem ispravno implementira definisanu poslovnu logiku i bezbjednosne mehanizme. Svi testni slučajevi su uspješno izvršeni, čime je verifikovana tačnost finansijskih proračuna, pouzdanost revizorskog traga i ispravnost kontrole pristupa.

Dobijeni rezultati pokazuju da razvijeni sistem ispunjava postavljene funkcionalne i nefunkcionalne zahtjeve, te da predstavlja stabilnu osnovu za dalju nadogradnju i primjenu u realnom radnom okruženju.

Pored funkcionalne verifikacije testnim slučajevima, izvršena je i analiza pokrivenosti koda testovima, čime je dobijen dodatni kvantitativni pokazatelj obuhvata testiranja. Na slici 5.1 prikazan je rezultat mjerenja pokrivenosti koda alatom pytest-cov, pri čemu ukupna pokrivenost u evaluiranoj verziji sistema iznosi 64%, uz 74 uspješno izvršena testa bez neuspješnih slučajeva.

Dobijeni rezultat treba tumačiti u kontekstu strukture sistema. Najveća pokrivenost ostvarena je nad test modulima i ključnim djelovima domena i poslovne logike, uključujući

modele, validacione šeme i centralne servisne komponente za obradu uplata i dnevno zatvaranje, dok je manja pokrivenost zabilježena u pojedinim blueprint i repository modulima koji obuhvataju šire skupove API ruta i pomoćnih operacija. Istovremeno, fajlovi poput seed.py i wsgi.py imaju zanemarljivu ili nultu pokrivenost, što je očekivano jer predstavljaju pomoćne ulazne i razvojne komponente, a ne jezgro poslovne logike sistema.

Na taj način metrika pokrivenosti ne predstavlja samo zbirni procenat, već i pokazatelj da su testovi prvenstveno usmjereni na dijelove sistema od najvećeg značaja za integritet finansijskih podataka, bezbjednost i konzistentnost izvršavanja poslovnih procesa. U kombinaciji sa uspješno izvršenim testnim scenarijima, ovakav rezultat potvrđuje da razvijeni sistem posjeduje solidan nivo verifikacije i predstavlja stabilnu osnovu za dalju nadogradnju i primjenu u realnom radnom okruženju.

```
tests\__init__.py                                  0      0   100%
tests\conftest.py                                101      5    95%   95-103
tests\test_01_infrastructure.py                   15      0   100%
tests\test_02_payment_logic.py                   209      0   100%
tests\test_03_daily_close_logic.py               131      0   100%
tests\test_04_tips_and_payouts.py                 80      4    95%   90, 142-144
tests\test_05_audit_security.py                   28      0   100%
tests\test_06_shared_terminal.py                  32      0   100%
tests\test_07_auth_schemas.py                     31      0   100%
tests\test_08_auth_routes.py                      66      0   100%
tests\test_09_management_logic.py                 85      2    98%   101, 182
tests\test_10_data_safety.py                      44      1    98%   55
tests\test_11_edge_cases.py                       75      1    99%   136
tests\test_12_api_permissions.py                 122      1    99%   37
tests\test_13_search_and_pagination.py            74      0   100%
wsgi.py                                            4      4     0%   1-6
--------------------------------------------------------------------------------
TOTAL                                           5706   2063    64%
Coverage HTML written to dir htmlcov
========================================================================================== 74 passed
```

Slika 5.1: Rezultat mjerenja pokrivenosti koda testovima

6 ZAKLJUČAK

U okviru ovog diplomskog rada projektovan je i implementiran serverski softverski sistem za upravljanje finansijskim tokovima u privatnim medicinskim ustanovama, sa posebnim fokusom na integritet podataka, konkurentni pristup i podršku za višekorisnički i multi-tenant rad. Polazna pretpostavka rada bila je da postojeća rješenja često ne pružaju adekvatne mehanizme za pouzdano finansijsko evidentiranje u okruženjima sa više zaposlenih i zajedničkim radnim stanicama, što može dovesti do grešaka i nekonzistentnosti podataka.

Razvijeni sistem demonstrira da je moguće ostvariti visok nivo pouzdanosti i sljedivosti finansijskih operacija primjenom relacijskog modela podataka, jasno definisanih transakcionih pravila i stroge izolacije podataka između klinika. Posebna pažnja posvećena je modelovanju finansijskih tokova, uključujući planove otplate, djelimične uplate, napojnice i dnevno zatvaranje kase, čime su obuhvaćeni realni poslovni scenariji iz svakodnevne prakse zdravstvenih ustanova.

Arhitektura sistema zasnovana na slojevitom pristupu omogućila je jasno razdvajanje odgovornosti između sloja pristupa podacima, sloja poslovne logike i sloja aplikacionog interfejsa. Ovakva organizacija koda doprinosi boljoj održivosti sistema, lakšem testiranju i mogućnosti budućeg proširenja funkcionalnosti bez narušavanja postojećih komponenti.

Poseban doprinos rada ogleda se u implementaciji mehanizama revizorskog traga i sekundarne autorizacije, koji omogućavaju pouzdano evidentiranje svih osjetljivih operacija, čak i u scenarijima zajedničkog korišćenja terminala. Na ovaj način obezbijeđena je jasna odgovornost korisnika i mogućnost naknadne analize događaja, što je od ključnog značaja za finansijske sisteme.

Ispravnost i stabilnost sistema verifikovani su kroz sveobuhvatno testiranje, koje obuhvata poslovnu logiku, bezbjednosne mehanizme i granične slučajeve. Testovi potvrđuju da sistem korektno reaguje u različitim realnim i izuzetnim scenarijima, čime se dodatno potvrđuje pouzdanost predloženog rješenja.

Mogući pravci budućeg razvoja uključuju uvođenje naprednijih mehanizama izvještavanja, optimizaciju performansi za veće opterećenje, kao i integraciju sa eksternim zdravstvenim ili računovodstvenim sistemima. Takođe, prelazak na produkcionu bazu podataka i proširenje testiranja na veće skupove podataka omogućili bi dodatnu validaciju sistema u uslovima stvarnog rada.

Na osnovu ostvarenih rezultata može se zaključiti da razvijeni sistem predstavlja stabilnu i skalabilnu osnovu za upravljanje finansijskim poslovanjem manjih i srednjih medicinskih ustanova, uz visok nivo bezbjednosti, integriteta podataka i prilagodljivosti različitim organizacionim zahtjevima.

7 LITERATURA

[1] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[2] PostgreSQL Global Development Group, "Transaction Isolation," PostgreSQL Documentation (current). Accessed: 2025-12-25. Available: https://www.postgresql.org/docs/current/transaction-iso.html

[3] PostgreSQL Global Development Group, "Explicit Locking," PostgreSQL Documentation (current). Accessed: 2025-12-25. Available: https://www.postgresql.org/docs/current/explicit-locking.html

[4] PostgreSQL Global Development Group, "SELECT," PostgreSQL Documentation (current). Accessed: 2025-12-25. Available: https://www.postgresql.org/docs/current/sql-select.html

[5] PostgreSQL Global Development Group, "Concurrency Control," PostgreSQL Documentation (current). Accessed: 2025-12-25. Available: https://www.postgresql.org/docs/current/mvcc.html

[6] Internet Engineering Task Force (IETF), "JSON Web Token (JWT)," RFC 7519, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7519

[7] Pallets Projects, "Flask Documentation," (current). Accessed: 2025-12-25. Available: https://flask.palletsprojects.com/

[8] SQLAlchemy Authors, "SQLAlchemy 2.0 Documentation," (current). Accessed: 2025-12-25. Available: https://docs.sqlalchemy.org/en/20/

[9] Alembic Authors, "Alembic Documentation," (current). Accessed: 2025-12-25. Available: https://alembic.sqlalchemy.org/

[10] Pydantic Authors, "Pydantic Documentation (v2)," (current). Accessed: 2025-12-25. Available: https://docs.pydantic.dev/

[11] Flask-JWT-Extended Contributors, "Flask-JWT-Extended Documentation," (current). Accessed: 2025-12-25. Available: https://flask-jwt-extended.readthedocs.io/

[12] Docker Inc., “Docker Documentation,” (current). Accessed: 2025-12-25. Available: https://docs.docker.com/

[13] Docker Inc., “Docker Compose Documentation,” (current). Accessed: 2025-12-25. Available: https://docs.docker.com/compose/